EV Charging Data Streaming and Analysis

- COMP5339 Project Assignment 2 -

In this data engineering assignment, your team will retrieve, process and visualise simulated real-time EV charger pricing and usage data. The project aims to provide practical experience in core data engineering skills such as data retrieval, stream processing, integration, cleaning, transformation, and visualisation.

This is a **group assignment**, comprising teams of 3 to 4 students. By default, you will remain in **the same group as formed for Assignment 1**, but you are free to switch groups (in Assignment groups setting) if you prefer.

Submit your Assignment 2 report & code by **23:59, Sunday 1 November (Week 12)**.
All submissions will undergo plagiarism checks.

Assignment 2 Tasks

This Assignment 2 concentrates on acquiring and visualizing simulated real-time data about EV charging in NSW. Your team needs to develop Python programs to complete the following tasks:

1. **Synthetic Dataset Generation**
   We provide you a dataset generator script that generates **simulated dynamic pricing** and **charging sessions** for a list of EV chargers. This dataset generator bases its simulated prices on real AEMO NSW wholesale electricity prices for a chosen month.

   Your first task is to use this dataset generator to generate a synthetic dataset of hourly dynamic prices for **your list of EV chargers from Assignment 1**. The script also generates a sequence of simulated charging events at those stations, with session-start, session-end and power meter events of an active charging session. The generated datasets will serve as input to Task 2.

2. **Data Publishing via MQTT**
   For each new **`price_update`**, **`session_start`**, **`session_end`**, and **`meter`** event record of a charger in the time series, publish a message to a MQTT server containing all the necessary information required for subsequent tasks, simulating an ordered data stream (event time order). Design appropriate JSON message formats and only publish new events (i.e. actual price changes, not repeating same price). Use at least a **0.1**-second delay between each published message.

3. **Data Subscription and Visualisation**
   Create a map-based dashboard using a Python framework of your choice (similar to https://plugshare.com or https://plugroo.com.au).

Specifically, subscribe to the MQTT messages published in the previous Task 2. For each received message, dynamically add a marker to the map at the corresponding charger's location (as collected in Assignment 1). Each marker must display at least one dynamically updated attribute (e.g. price or occupancy). Additional dynamic attributes may be shown through tooltips, pop-ups, or user controls. When a user clicks a marker, it should display a pop-up with the charger's details such as name (if applicable), address, plug types, operator network and the latest price. For full marks: If there is an active charging session on that charger, show a live-updated line curve of the charging session's progress as reported by the `meter` events.

Optional (not assessed): Add filtering by operator, charger type, or charger ratings.

4. **Stateful Event Storage and Charging Session Analytics**

   Store all MQTT events received in Task 3 in DuckDB and integrate them with your charger database from Assignment 1 (see marking rubric for details).

   Your stored data should support efficient querying and analysis of both historical charging activity and currently active charging sessions, and the calculation of historical charger usage statistics. Extend your dashboard to display at least one current-state metric (e.g. active charging sessions) and one historical metric derived from the stored event data.

5. **Documentation and Reporting**

   Maintain detailed documentation throughout your project workflow. Your final report should clearly summarise your methods for data generation, MQTT publishing and subscription, database integration, data analysis, and live visualisation. Specifically:

   - Explain your data generation for your specific EV charger list.
   - Explain any steps required to prepare the time-series data for publishing as individual MQTT messages.
   - Explain the system components and their roles for your dashboard that displays continuous charger usage and/or pricing data.
   - Explain how you mapped the synthetic price update and charging session events to your EV charger dataset from Assignment 1 and its visualization on the map.
   - Document your schema changes and any physical design optimisations.
   - Include at least one screenshot of your running dashboard in the report.
   - Describe challenges encountered and how you overcame them.
   - Provide recommendations for future improvements.

- **Recommended Report Sections**
  - Introduction
  - Synthetic Event Generation
  - MQTT Publishing
  - MQTT Subscription and Event Processing
  - Event Storage and Integration with Assignment 1 Database
  - Dashboard and Visualisation
  - Challenges and Limitations
  - Team Contributions

Deliverables

There will be a group demo in Week 12/13.

Besides, submit the following three items via Canvas:

1. Source Code and Database Package (**.zip**)
   with all data, scripts, and Python files (**.py** or **.ipynb**), **requirements.txt** and **README.md**
   - Organise your code and Python notebooks in a clear and logical project structure.
   - Include a **requirements.txt** file listing all Python packages required to run your program within a clean Python virtual environment.
   - Include your **generated datasets and event streams** from Task 1
   - Include your updated **DuckDB database** file from Task 4 plus a **DDL script**.
   - Include a **README** file (using Markdown format) containing:
     - project structure,
     - setup instructions,
     - execution instructions,
     - assumptions made,
     - description of outputs produced and any included data files.
2. Project Report (**.pdf**)
   - Provide a clearly structured report of **up to 6 pages** of content. The title page and any optional appendix do not count towards the page limit. The title page must show your group name and list the team members by name and SID. See Task 5 above for recommended report sections.
   - Your report must include a short paragraph, not exceeding 100 words, detailing the contribution of each team member, in the same format as in Assignment 1.
3. Generative AI (genAI) and Automated Writing Tools Usage Report
   - The use of genAI tools is allowed in this assignment to support the development of your code and report – as long as the essence is your contribution. You remain responsible for verifying the correctness and originality of all submitted work.
   - You must acknowledge any genAI tools usage by submitting a "Generative AI and Automated Writing Tools Usage Report".

Submission Deadline

All Assignment 2 deliverables are due by **23:59, Sunday 1 November (Week 12).** Late submissions will incur penalties according to university policy.

Group Member Participation and Marking

This is a group assignment. Please declare the level of participation of each team member as part of your report (see 'Deliverables' above), but also note that the mark awarded for your assignment is conditional on you being able to explain any aspect of the submitted work to your tutor during the team demo or the lecturer if asked.

If your group is experiencing difficulties with the content, you should ask on Ed (use a private post if you need to discuss code or report writing directly).

Level of contribution	Proportion of group mark received
No participation.	0%
Minor contributor with at least some understanding of the group submission.	50%
Major contributor to the group's submission.	100%